Operating Systems – Code Collection

Formatted source code provided in the uploaded file

Fcfs

```
#include <stdio.h>
```

Int Main()

```
{
    int n, i, time = 0;
    int at[10], bt[10], ct[10], tat[10], wt[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Arrival Time and Burst Time:\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    for(i = 0; i < n; i++)
    {
        if(time < at[i])
            time = at[i];

        time = time + bt[i];
        ct[i] = time;

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        avgwt += wt[i];
        avgtat += tat[i];
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage Waiting Time = %.2f", avgwt/n);
    printf("\nAverage Turnaround Time = %.2f", avgtat/n);

    return 0;
}
```

Round Robin

```
#include <stdio.h>
```

Int Main()

```
{
    int n, i, tq, time = 0, flag;
    int at[10], bt[10], rt[10], wt[10], tat[10], ct[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    while(1)
    {
        flag = 0;

        for(i = 0; i < n; i++)
        {
            if(rt[i] > 0 && at[i] <= time)
            {
                flag = 1;

                if(rt[i] > tq)
                {
                    time = time + tq;
                    rt[i] = rt[i] - tq;
                }
                else
                {
                    time = time + rt[i];
                    rt[i] = 0;
                    ct[i] = time;
                }
            }
        }

        if(flag == 0)
        {
            int finished = 1;

            for(i = 0; i < n; i++)
            {
                if(rt[i] > 0)
                    finished = 0;
            }

            if(finished == 1)
                break;

            time++;
        }
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

Sjf

```
#include <stdio.h>
```

Int Main()

```
{
    int n, i, time = 0, count = 0, min, pos;
    int at[10], bt[10], ct[10], tat[10], wt[10], done[10] = {0};
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    while(count < n)
    {
        min = 999;
        pos = -1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= time && done[i] == 0)
            {
                if(bt[i] < min)
                {
                    min = bt[i];
                    pos = i;
                }
            }
        }

        if(pos == -1)
        {
            time++;
        }
        else
        {
            time = time + bt[pos];
            ct[pos] = time;
            tat[pos] = ct[pos] - at[pos];
            wt[pos] = tat[pos] - bt[pos];

            done[pos] = 1;
            count++;

            avgwt += wt[pos];
            avgtat += tat[pos];
        }
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage WT = %.2f", avgwt/n);
    printf("\nAverage TAT = %.2f", avgtat/n);

    return 0;
}
```

Srtf

```
#include <stdio.h>
```

Int Main()

```
{
    int n, i, time = 0, count = 0, min, pos;
    int at[10], bt[10], rt[10], ct[10], tat[10], wt[10];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT and BT for P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }

    while(count < n)
    {
        min = 999;
        pos = -1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= time && rt[i] > 0)
            {
                if(rt[i] < min)
                {
                    min = rt[i];
                    pos = i;
                }
            }
        }

        if(pos == -1)
        {
            time++;
        }
        else
        {
            rt[pos]--;
            time++;

            if(rt[pos] == 0)
            {
                ct[pos] = time;
                count++;
            }
        }
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        avgwt += wt[i];
        avgtat += tat[i];
    }

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage WT = %.2f", avgwt/n);
    printf("\nAverage TAT = %.2f", avgtat/n);

    return 0;
}
```

Priority Non Preamtion

```
#include <stdio.h>
```

Int Main()

```
{
    int n, i, time = 0, count = 0;
    int at[10], bt[10], pr[10], ct[10], tat[10], wt[10];
    int done[10] = {0}, min, pos;
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT, BT and Priority for P%d: ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
    }

    while(count < n)
    {
        min = 999;
        pos = -1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= time && done[i] == 0)
            {
                if(pr[i] < min)
                {
                    min = pr[i];
                    pos = i;
                }
            }
        }

        if(pos == -1)
        {
            time++;
        }
        else
        {
            time = time + bt[pos];
            ct[pos] = time;

            tat[pos] = ct[pos] - at[pos];
            wt[pos] = tat[pos] - bt[pos];

            done[pos] = 1;
            count++;

            avgwt += wt[pos];
            avgtat += tat[pos];
        }
    }

    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage WT = %.2f", avgwt/n);
    printf("\nAverage TAT = %.2f", avgtat/n);

    return 0;
}
```

Priority Preamtion

```
#include <stdio.h>
```

Int Main()

```
{
    int n, i, time = 0, count = 0;
    int at[10], bt[10], pr[10], rt[10];
    int ct[10], tat[10], wt[10];
    int min, pos;
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter AT, BT and Priority for P%d: ", i+1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        rt[i] = bt[i];
    }

    while(count < n)
    {
        min = 999;
        pos = -1;

        for(i = 0; i < n; i++)
        {
            if(at[i] <= time && rt[i] > 0)
            {
                if(pr[i] < min)
                {
                    min = pr[i];
                    pos = i;
                }
            }
        }

        if(pos == -1)
        {
            time++;
        }
        else
        {
            rt[pos]--;
            time++;

            if(rt[pos] == 0)
            {
                ct[pos] = time;
                count++;
            }
        }
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        avgwt += wt[i];
        avgtat += tat[i];
    }

    printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
    }

    printf("\nAverage WT = %.2f", avgwt/n);
    printf("\nAverage TAT = %.2f", avgtat/n);

    return 0;
}
```

Producer _Consumer

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5

int buffer[N];
int in = 0, out = 0;

sem_t empty, full, mutex;

void *producer()
{
    int item;

    for(item = 1; item <= 5; item++)
    {
        sem_wait(&empty);
        sem_wait(&mutex);

        buffer[in] = item;
        printf("Producer produced: %d\n", item);

        in = (in + 1) % N;

        sem_post(&mutex);
        sem_post(&full);
    }

    return NULL;
}

void *consumer()
{
    int item, i;

    for(i = 1; i <= 5; i++)
    {
        sem_wait(&full);
        sem_wait(&mutex);

        item = buffer[out];
        printf("Consumer consumed: %d\n", item);

        out = (out + 1) % N;

        sem_post(&mutex);
        sem_post(&empty);
    }

    return NULL;
}
```

Int Main()

```
{
    pthread_t p, c;

    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);

    return 0;
}
```

Readers Writers

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t mutex, wrt;
int readcount = 0;
int data = 10;

void *reader(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&mutex);
    readcount++;

    if(readcount == 1)
        sem_wait(&wrt);

    sem_post(&mutex);

    printf("Reader %d is reading data: %d\n", id, data);

    sem_wait(&mutex);
    readcount--;

    if(readcount == 0)
        sem_post(&wrt);

    sem_post(&mutex);

    return NULL;
}

void *writer(void *arg)
{
    int id = *(int *)arg;

    sem_wait(&wrt);

    data++;
    printf("Writer %d is writing data: %d\n", id, data);

    sem_post(&wrt);

    return NULL;
}
```

Int Main()

```
{
    pthread_t r[2], w[2];
    int id[4] = {1, 2, 1, 2};

    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);

    pthread_create(&r[0], NULL, reader, &id[0]);
    pthread_create(&r[1], NULL, reader, &id[1]);

    pthread_create(&w[0], NULL, writer, &id[2]);
    pthread_create(&w[1], NULL, writer, &id[3]);

    pthread_join(r[0], NULL);
    pthread_join(r[1], NULL);
    pthread_join(w[0], NULL);
    pthread_join(w[1], NULL);

    sem_destroy(&mutex);
    sem_destroy(&wrt);

    return 0;
}
```

Bankers

```
#include <stdio.h>
```

Int Main()

```
{
    int n, m, i, j, k;
    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], finish[10] = {0}, safe[10];
    int count = 0, found;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("Enter Allocation matrix:\n");

    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Maximum matrix:\n");

    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available resources:\n");

    for(j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    // Calculate Need matrix
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    // Safety algorithm
    while(count < n)
    {
        found = 0;

        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                for(j = 0; j < m; j++)
                {
                    if(need[i][j] > avail[j])
                        break;
                }

                if(j == m)
                {
                    for(k = 0; k < m; k++)
                        avail[k] += alloc[i][k];

                    safe[count] = i;
                    count++;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0)
            break;
    }

    if(count == n)
    {
        printf("\nSystem is in SAFE STATE\n");
        printf("Safe Sequence: ");

        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    }
    else
    {
        printf("\nSystem is in UNSAFE STATE");
    }
    return 0;
```

```
}
```

Pipe Full Duplex

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
```

Int Main()

```
{
    int p1[2], p2[2];
    int pid;
    char msg1[] = "Hello Child";
    char msg2[] = "Hello Parent";
    char buffer[50];

    pipe(p1);
    pipe(p2);

    pid = fork();

    if(pid == 0)
    {
        // Child Process
        close(p1[1]);
        close(p2[0]);

        read(p1[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);

        write(p2[1], msg2, strlen(msg2) + 1);

        close(p1[0]);
        close(p2[1]);
    }
    else
    {
        // Parent Process
        close(p1[0]);
        close(p2[1]);

        write(p1[1], msg1, strlen(msg1) + 1);

        read(p2[0], buffer, sizeof(buffer));
        printf("Parent received: %s\n", buffer);

        close(p1[1]);
        close(p2[0]);
    }

    return 0;
}
```

Fifo

```
#include <stdio.h>
```

Int Main()

```
{
    int n, f, i, j, k;
    int pages[50], frames[10];
    int index = 0, fault = 0, hit;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
        frames[i] = -1;

    for(i = 0; i < n; i++)
    {
        hit = 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                hit = 1;
                break;
            }
        }

        if(hit == 0)
        {
            frames[index] = pages[i];
            index = (index + 1) % f;
            fault++;
        }

        printf("\n");

        for(k = 0; k < f; k++)
        {
            if(frames[k] == -1)
                printf("- ");
            else
                printf("%d ", frames[k]);
        }
    }

    printf("\nTotal Page Faults = %d\n", fault);
    printf("Total Page Hits = %d\n", n - fault);

    return 0;
}
```

Lru

```
#include <stdio.h>
```

Int Main()

```
{
    int n, f, i, j, k;
    int pages[50], frames[10], time[10];
    int count = 0, fault = 0, hit, pos, min;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    for(i = 0; i < n; i++)
    {
        hit = 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                hit = 1;
                time[j] = ++count;
                break;
            }
        }

        if(hit == 0)
        {
            pos = -1;

            for(j = 0; j < f; j++)
            {
                if(frames[j] == -1)
                {
                    pos = j;
                    break;
                }
            }

            if(pos == -1)
            {
                min = time[0];
                pos = 0;

                for(j = 1; j < f; j++)
                {
                    if(time[j] < min)
                    {
                        min = time[j];
                        pos = j;
                    }
                }
            }

            frames[pos] = pages[i];
            time[pos] = ++count;
            fault++;
        }

        printf("\n");

        for(k = 0; k < f; k++)
        {
            if(frames[k] == -1)
                printf("- ");
            else
                printf("%d ", frames[k]);
        }
    }
    printf("\nTotal Page Faults = %d\n", fault);
```

```
    printf("Total Page Hits = %d\n", n - fault);

    return 0;
}
```